

Curso online: **Programación orientada a objetos: Java avanzado**

Ejercicio 1

Autores

Aurelio García Prieto

Miguel Sallent Sánchez

ÍNDICE

1. Objetivo.....	3
2. Primera parte: extracción de la información.....	3
3. Segunda parte: almacenamiento en base de datos.....	5

1. Objetivo

El objetivo de este ejercicio es realizar una aplicación que extraiga cierta información de una página web en Internet y la almacene en una base de datos.

2. Primera parte: extracción de la información

La aplicación se conectará a la siguiente url:

https://taferk.github.io/CursoJavaAvanzado/site/ejercicio1_ibex35.html

En esta página hay una tabla con unas cotizaciones de las acciones de las 35 empresas que forman el IBEX 35:

COTIZACIONES DEL IBEX 35				
Esta página simula una tabla con las cotizaciones de las acciones del IBEX 35 a la fecha actual. (todos los precios mostrados son ficticios)				
IBEX 35				
Acción	Última cotización	Anterior	Variación	Fecha
ACCIONA	35.67€	35.63€	0.1%	06-08-2025 11:41:57
ACCIONA ENER	1.53€	1.53€	0.3%	06-08-2025 11:41:57
ACERINOX	56.09€	57.35€	-2.2%	06-08-2025 11:41:57
ACS	88.84€	90.75€	-2.1%	06-08-2025 11:41:57
AENA	1.79€	1.76€	1.6%	06-08-2025 11:41:57
AMADEUS	15.3€	15.41€	-0.7%	06-08-2025 11:41:57
ARCELORMIT.	58.81€	59.65€	-1.4%	06-08-2025 11:41:57

El alumno debe realizar una clase que se conecte a dicha url y extraiga, para cada uno de los 35 valores del IBEX 35:

- El nombre de la empresa
- El valor de la última cotización (columna “Última cotización”)

Se deberá analizar el HTML y procesarlo. Se pueden utilizar expresiones regulares o bien cualquier otro método (búsqueda de cadenas de texto, subcadenas, split, etc).

En esta primera parte del ejercicio bastará con mostrar la información por consola, para ver que la información se extrae correctamente.

Uso de expresiones regulares

Si el alumno lo desea puede utilizar [expresiones regulares](#) para realizar el ejercicio. Las expresiones regulares son un mecanismo muy potente para buscar textos o comprobar que determinados textos cumplen ciertos patrones.

No es el objetivo del curso estudiar las expresiones regulares, así pues a continuación mostramos algunas orientaciones para los alumnos que deseen utilizarlas (los alumnos que lo prefieran podrán utilizar cualquier otro método).

Por ejemplo, el algoritmo podría ser:

1. Quitar todos los saltos de línea, de forma que tengamos todo el html en una sola línea.
2. Extraer el tbody de la tabla cuyo id es "listado_valores", usando la siguiente expresión regular:

```
Pattern tableBodyPattern =  
    Pattern.compile("<table  
id=\"id-listado-acciones\".+?<tbody>(.*?)</tbody>.+?</table>");
```

La clave aquí está en la secuencia ".+?", que quiere decir:

- el punto indica "cualquier carácter" excepto salto de línea
- el signo + indica "una o más veces"
- el interrogante quiere decir *lazy* (en oposición a *greedy*), lo cual quiere decir que intentará el match con el menor número posible de caracteres. Es decir, si tuviéramos dos tablas seguidas, con la opción greedy nos tomaría el tbody de la segunda tabla, con la opción lazy nos toma el primer tbody.

Para ejecutar la expresión regular creamos un Matcher:

```
Matcher tableBodyMatcher = tableBodyPattern.matcher(html.toString());
```

Donde "html" es el texto sin saltos de líneas, como ya hemos comentado.

A continuación podemos utilizar los siguientes métodos:

- `tableBodyMatcher.find()`: devolverá true si el texto cumple la expresión regular
 - `tableBodyMatcher.group(1)`: devolverá el primer grupo. Los grupos son los bloques que hemos puesto entre paréntesis, en este caso sólo hay un grupo y es el interior del tbody.
3. Una vez tenemos el tbody debemos extraer el contenido de las filas. Por ejemplo podemos usar la siguiente expresión regular:

```
Pattern trPattern =  
    Pattern.compile("<tr>.+?<td.+?>(.*?)</td>.+?<td.+?><span.+?>(.*?)€</span></td>.  
.+?</tr>");
```

Observamos que es la estructura de una fila, e incluimos dos grupos entre paréntesis, uno para el nombre de la empresa y otro para el valor de la cotización. En este caso crearemos un matcher con el tbody que

hemos extraído en el paso anterior y haremos un bucle “while trMatcher.find()”, extrayendo en cada iteración los grupos 1 y 2 usando el método group del matcher.

3. Segunda parte: almacenamiento en base de datos

El alumno creará una base de datos SQLite, con únicamente una tabla llamada COTIZACIONES.

La tabla tendrá dos campos, NOMBRE y ULTIMO, ambos de tipo texto para simplificar.

Tanto la base de datos como la tabla no es necesario que se creen desde el programa java, pueden crearse desde la aplicación DB Browser.

El programa realizará lo siguiente:

- Obtendrá la información de las cotizaciones tal y como se ha indicado en la primera parte.
- A continuación borrará todas las filas de la tabla (SQL: “DELETE FROM COTIZACIONES”).
- A continuación insertará una fila por cada uno de los 35 valores leídos.

Tras la ejecución del programa, el alumno comprobará desde DB Browser que los 35 valores se han cargado correctamente.

El alumno entregará el código fuente (proyecto Eclipse) así como la base de datos creada, con los valores insertados, todo ello en un archivo zip o similar.

Estructura

Hoja de datos

Editar pragmas

Tabla:

COTIZACIONES

	NOMBRE	ULTIMO
	Filtro	Filtro
19	IAG	15.02
20	IBERDROLA	100.25
21	INDITEX	36.77
22	INDRA A	53.05
23	INM.COLONIAL	21.03
24	LOGISTA	36.62
25	MAPFRE	9.1
26	MERLIN	89.62
27	NATURGY	27.05
28	PUIG BRANDS	86.24
29	REDEIA	38.03
30	REPSOL	20.35
31	ROVI	23.79
32	SACYR	34.18
33	SOLARIA	12.29
34	TELEFONICA	7.68
35	UNICAJA	8.54

18 - 35 de 35